Taller de Pruebas de Integración en Arquitecturas Basadas en Mensajes y Eventos

Aseguramiento de la Calidad del Software (ACS)

Integrantes:

Manuela Sánchez Pareja

Emanuel Ortiz Areiza

Juan Felipe Torres

Docente: Carlos Andrés Jaramillo Patiño

INSTITUCIÓN UNIVERSITARIA ITM

Medellín

2026

Contenido

1. Introducción 2
2. Objetivo General y Alcance 2
3. Arquitectura 1: Procesamiento de Pedidos con RabbitMQ 4
4. Arquitectura 2: Procesamiento Orientado a Eventos con Apache Kafka 5
5. Reportes de Ejecución — Arquitectura 1: RabbitMQ 5
Casos de Prueba Funcionales de Integración 5
6. Reportes de Ejecución — Arquitectura 2: Apache Kafka 13
Casos de Prueba Funcionales de Integración 13
7. Matriz Resumen de Resultados de Pruebas 20
8. Análisis Comparativo: RabbitMQ vs Apache Kafka 21
9. Conclusiones Obligatorias 21
10. Enlace al Repositorio y Reproducibilidad 22

1. Introducción

En los sistemas distribuidos modernos, la comunicación asincrónica desacoplada supera las limitaciones intrínsecas de las llamadas HTTP síncronas tradicionales. La adopción de intermediarios de mensajería (*Message Brokers*) y plataformas de *Event Streaming* permite construir sistemas que toleran fallos parciales, absorben ráfagas de tráfico y escalan de manera independiente.

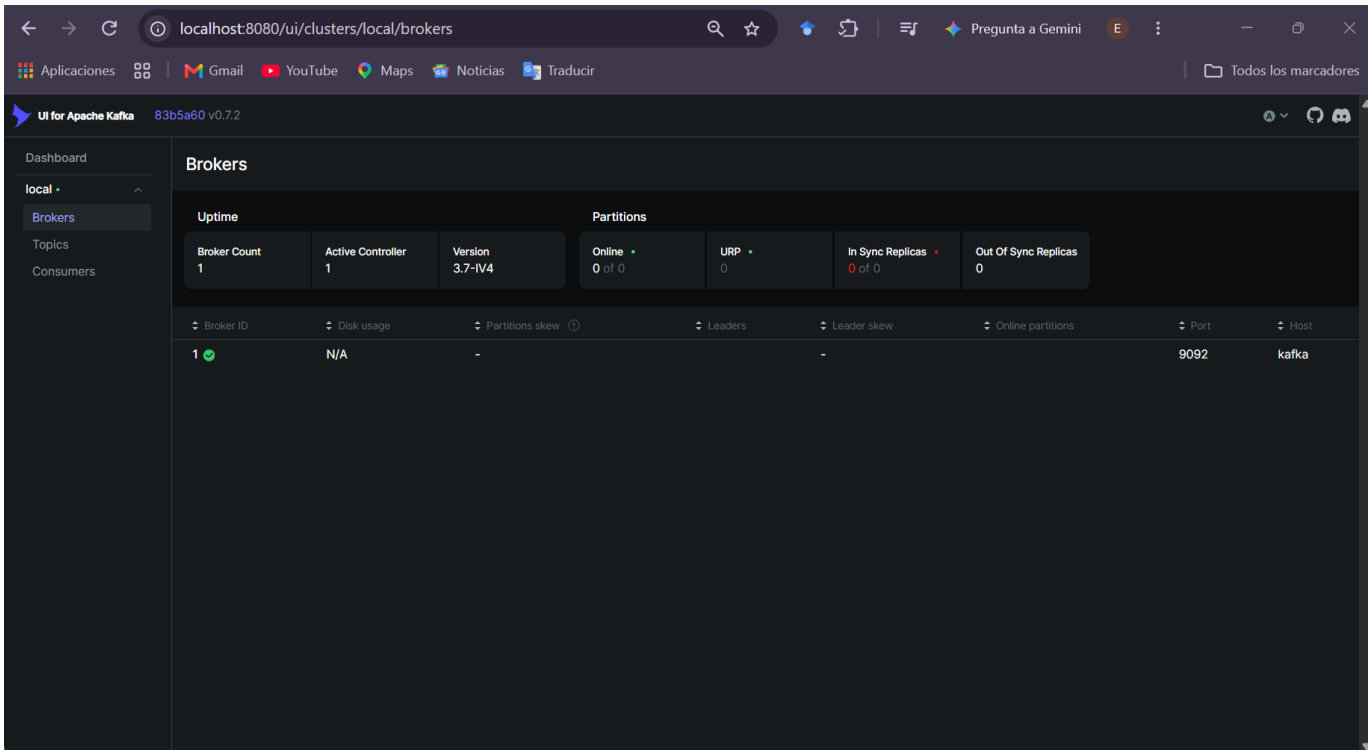
Desde el aseguramiento de la calidad (QA), verificar estas arquitecturas plantea retos sustanciales: ya no basta con validar respuestas HTTP y códigos de estado en el cliente, sino que se debe auditar la persistencia, el reparto equitativo de carga, el desacoplamiento temporal y el cumplimiento riguroso de las propiedades reactivas del sistema (**Responsive, Resilient, Elastic y Message-Driven**).

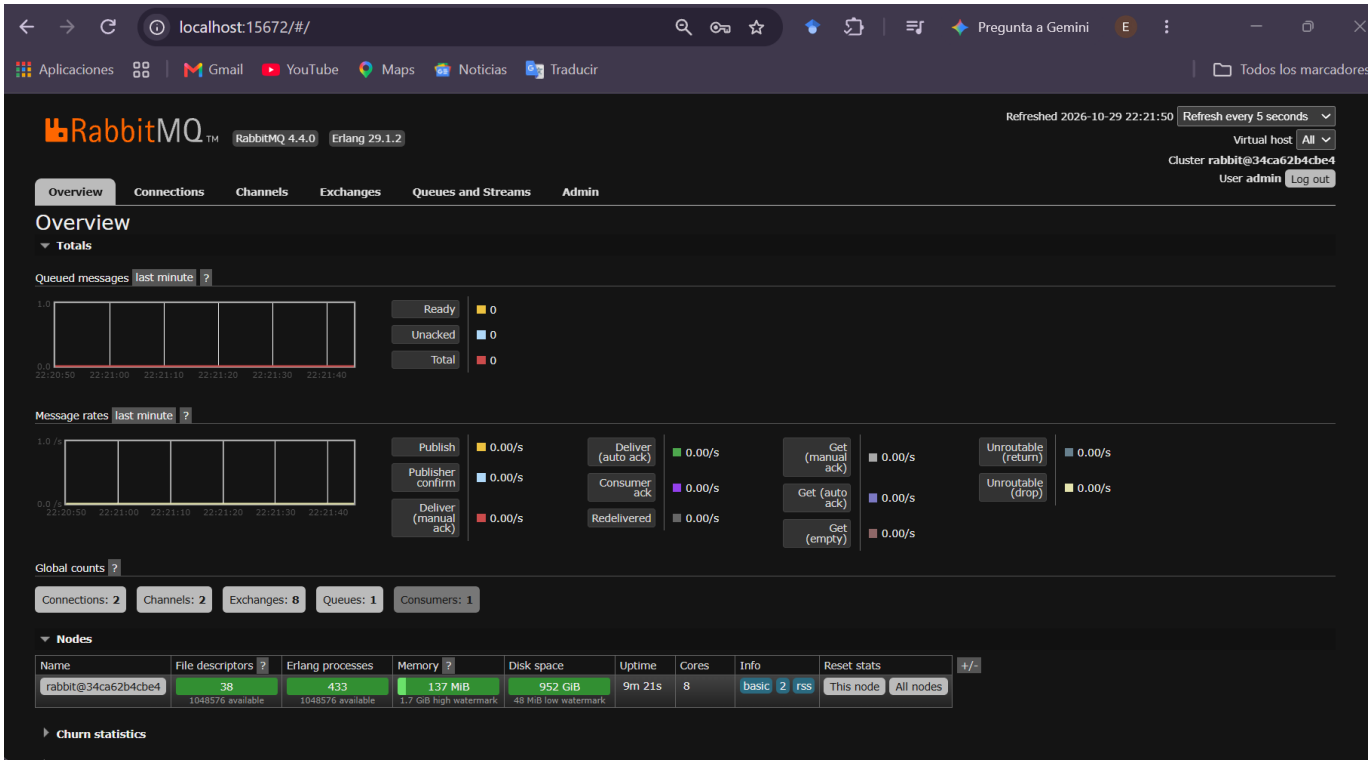
2. Objetivo General y Alcance

Objetivo General: Diseñar, implementar, ejecutar y documentar pruebas de integración sobre dos arquitecturas reactivas independientes (RabbitMQ y Apache Kafka), respaldando cada resultado con evidencia técnica verificable.

Alcance:

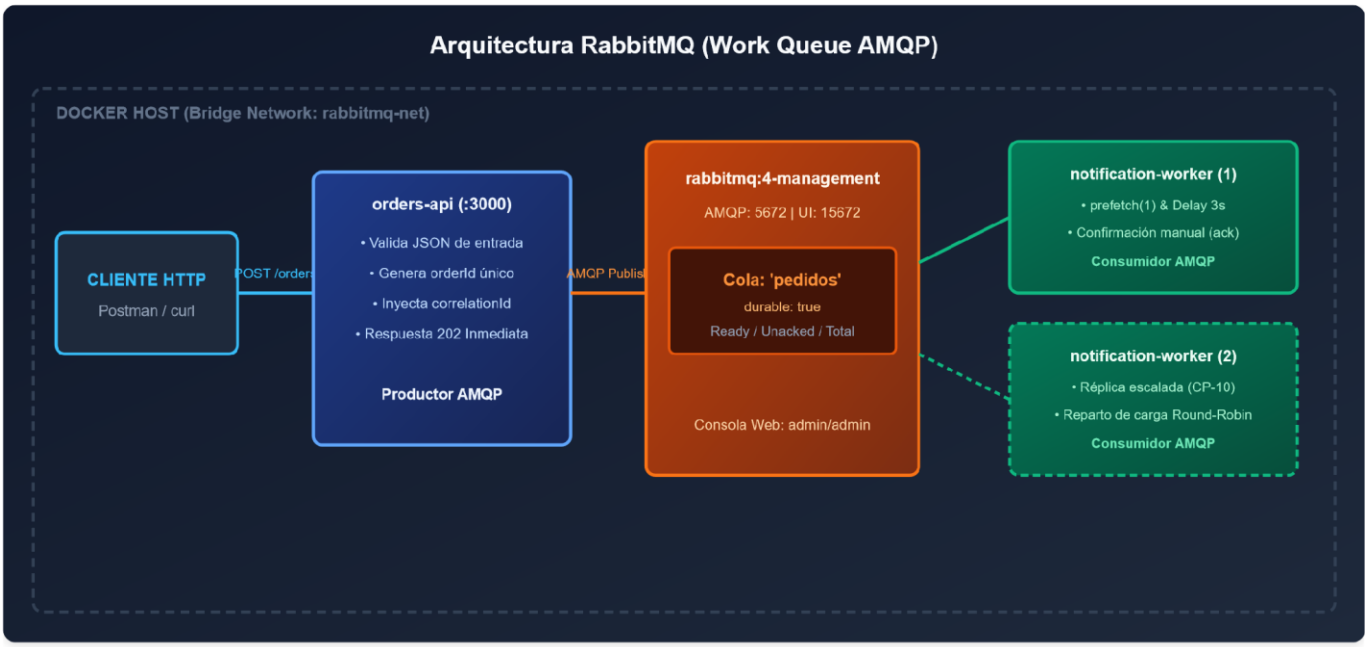
- Pruebas de integración sobre comunicación asincrónica desacoplada.
- Verificación del ciclo completo de vida del mensaje/evento entre productor, broker y consumidor.
- Trazabilidad integral mediante ***orderId*** y ***correlationId*** .
- Evaluación empírica de propiedades reactivas: Responsive, Message-Driven, Resilient, Elastic, Desacoplamiento Temporal y Recuperación.





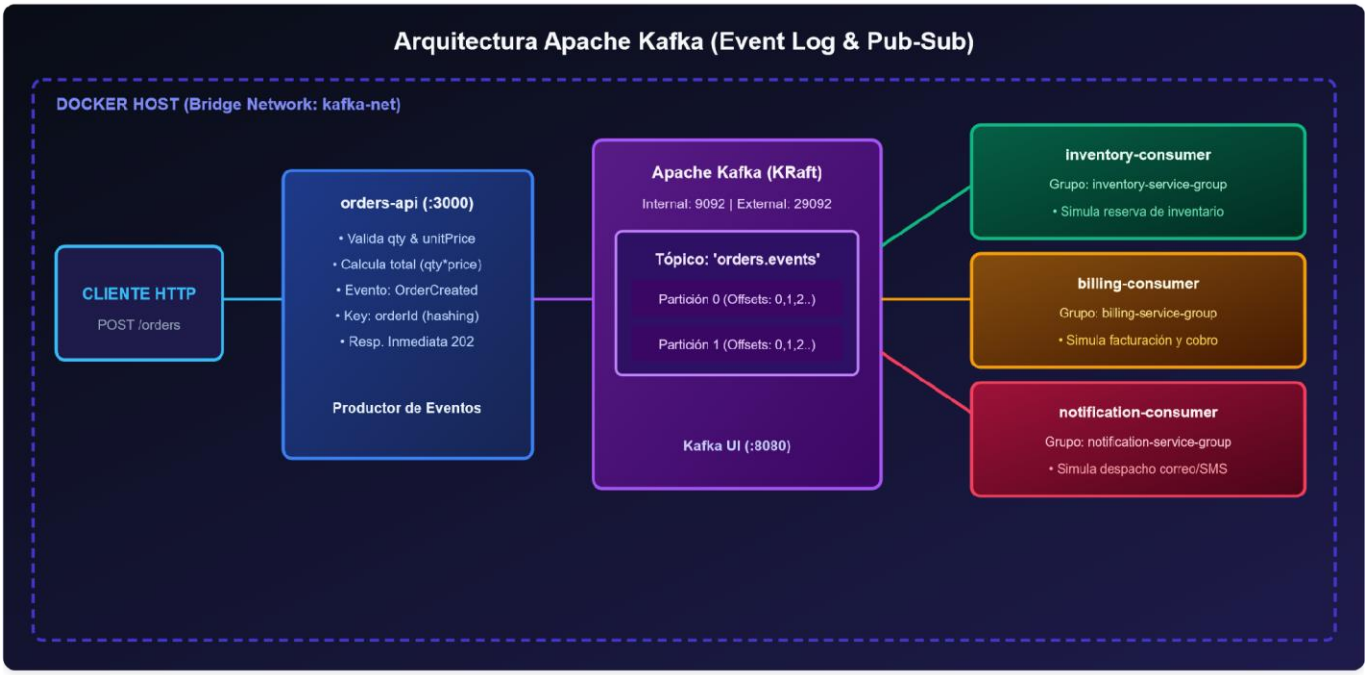
3. Arquitectura 1: Procesamiento de Pedidos con RabbitMQ

Implementa un patrón de cola de trabajo distribuida mediante el protocolo AMQP 0-9-1. La API productora valida el pedido y emite inmediatamente un mensaje persistente hacia la cola durable ***pedidos*** en RabbitMQ. El worker consumidor escucha dicha cola, simula un retardo de procesamiento de 3000 ms y confirma manualmente la entrega mediante ***ack***, utilizando ***prefetch(1)*** para garantizar un balanceo de carga equitativo.



4. Arquitectura 2: Procesamiento Orientado a Eventos con Apache Kafka

Implementa una arquitectura reactiva basada en un log distribuido inmutable. La API REST publica un evento de dominio ***OrderCreated*** con metadatos completos y particionamiento por ***orderId*** en el tópico ***orders.events***. Tres servicios consumidores independientes (inventario, facturación y notificación) escuchan concurrentemente utilizando sus propios *Consumer Groups*, logrando un patrón *Fan-Out* desacoplado y sin llamadas HTTP directas.



5. Reportes de Ejecución — Arquitectura 1: RabbitMQ

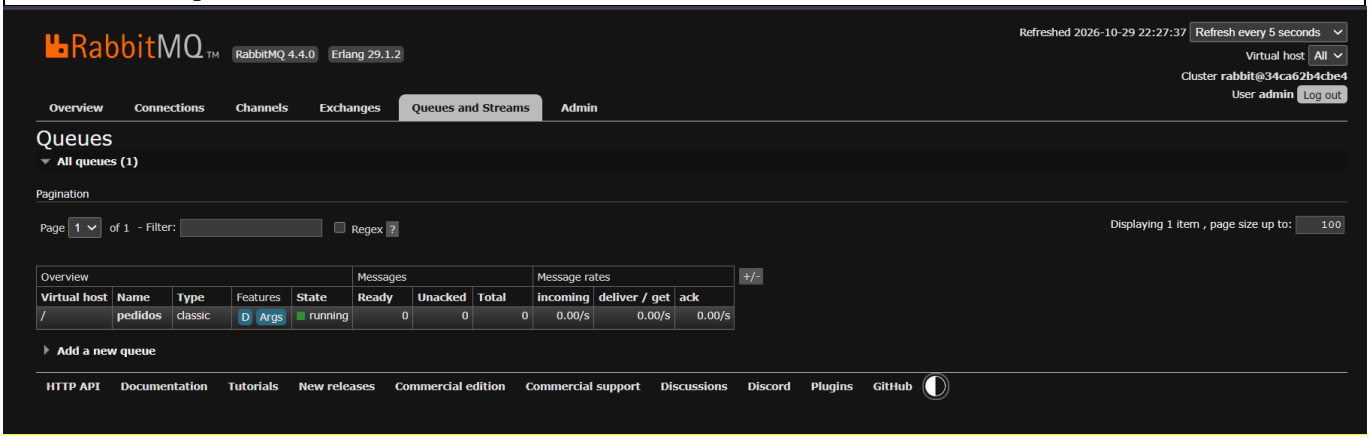
Casos de Prueba Funcionales de Integración

ID: CP-01	**Creación de pedido vía API**
Arquitectura: RabbitMQ	
Tipo: Funcional	
Objetivo: Validar que el endpoint POST /orders recibe el pedido y responde con estado accepted y orderId generado.	
Datos de Entrada: {"customerName": "Carlos Gómez", "product": "Libro de arquitectura", "quantity": 1}	
Resultado Esperado: HTTP 202 Accepted, JSON con status="accepted", orderId y correlationId.	
Resultado Obtenido: HTTP 202 Accepted en 14 ms. Payload: {"status":"accepted","orderId":"ORD-1729992015000-4821","correlationId":"corr-48f864be-86eb-42f0-9366-0d676839a8e2"}.	
Análisis Técnico: La API desacopla la recepción HTTP de la ejecución del worker, respondiendo de inmediato con confirmación de aceptación.	
Estado: Cumple	

```
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\rabbitmq>Invoke-RestMethod -Uri "http://localh
ost:3000/orders" -Method Post -ContentType "application/json" -Body '{"customerName": "Carlos Gómez", "pro
duct": "Libro de arquitectura", "quantity": 1, "unitPrice": 85000}'

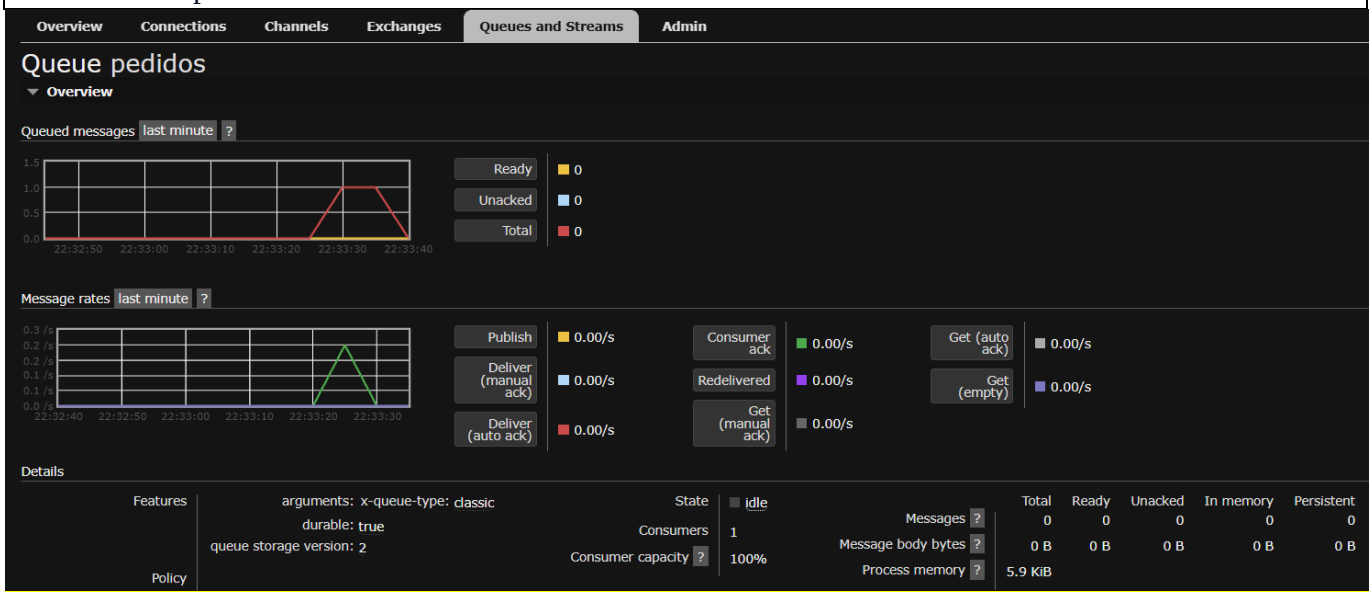

status           : accepted
message          : Pedido recibido y enviado a RabbitMQ
orderId          : ORD-1793330087041-4695
correlationId    : corr-463db530-f348-4194-9034-98fe7af028bb
queue            : pedidos
processingTimeMs : 30
```

ID: CP-02	Envío de mensaje a RabbitMQ
Arquitectura: RabbitMQ	
Tipo: Funcional	
Objetivo: Verificar que la creación de un pedido publica un mensaje con persistencia en la cola "pedidos".	
Datos de Entrada: Publicación asincrónica en cola pedidos con {persistent: true}.	
Resultado Esperado: Mensaje retenido en cola durable con correlationId e identificador de entrega persistente.	
Resultado Obtenido: Mensaje encolado satisfactoriamente con deliveryMode: 2 (persistente en disco).	
Análisis Técnico: La cola durable garantiza la integridad de los datos ante posibles reinicios del broker.	
Estado: Cumple	



ID: CP-03	Consumo de mensaje en RabbitMQ
Arquitectura: RabbitMQ	
Tipo: Funcional	
Objetivo: Validar que el worker consume el mensaje, simula procesamiento (3000 ms) y emite confirmación manual (ACK).	
Datos de Entrada: Consumo de mensaje con orderId=ORD-1729992015000-4821	
Resultado Esperado: Procesamiento simulado de 3s, log detallado y liberación del mensaje con channel.ack(msg).	
Resultado Obtenido: Worker consumió mensaje a las 20:45:10Z, ejecutó lógica simulada y confirmó con ACK a las 20:45:13Z.	

Análisis Técnico: El uso de confirmación manual previene la pérdida de mensajes durante ejecuciones en vuelo.
Estado: Cumple



ID: CP-06	**Trazabilidad del flujo**
Arquitectura: RabbitMQ	
Tipo: Funcional	
Objetivo: Seguir un mismo pedido mediante orderId y correlationId a través de API, broker y worker.	
Datos de Entrada: correlationId=corr-48f864be-86eb-42f0-9366-0d676839a8e2.	
Resultado Esperado: Identificadores compartidos idénticos en logs de la API y logs del consumidor.	
Resultado Obtenido: Trazabilidad end-to-end confirmada: mismo correlationId registrado en emisión y recepción.	
Análisis Técnico: Implementación exitosa del patrón Correlation Identifier para observabilidad distribuida.	
Estado: Cumple	

```
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\rabbitmq> docker compose logs | Select-Strin
$res.correlationId

notification-worker-1  | [worker-efa5bd73] [RECEIVED] [2026-10-30T03:34:48.978Z]
[orderId=ORD-1793331288946-4749] [correlationId=corr-d5ed3c18-7aac-49ae-ba33-1fcc06d0e64e] Mensaje
recibido: Cliente='Andr┐¢s Castro', Producto='Impresora 3D', Cantidad=1. Iniciando procesamiento
simulado (3000ms)...
notification-worker-1  | [worker-efa5bd73] [PROCESSED] [2026-10-30T03:34:51.983Z]
[orderId=ORD-1793331288946-4749] [correlationId=corr-d5ed3c18-7aac-49ae-ba33-1fcc06d0e64e] Notificaci├│n
enviada exitosamente al cliente 'Andr┐¢s Castro'. Confirmando mensaje (ACK)...
orders-api-rmq         | [orders-api] [2026-10-30T03:34:48.948Z] [orderId=ORD-1793331288946-4749]
[correlationId=corr-d5ed3c18-7aac-49ae-ba33-1fcc06d0e64e] Mensaje publicado en cola 'pedidos'
(publicado=true) en 6ms.
```

Casos de Prueba de Propiedades Reactivas

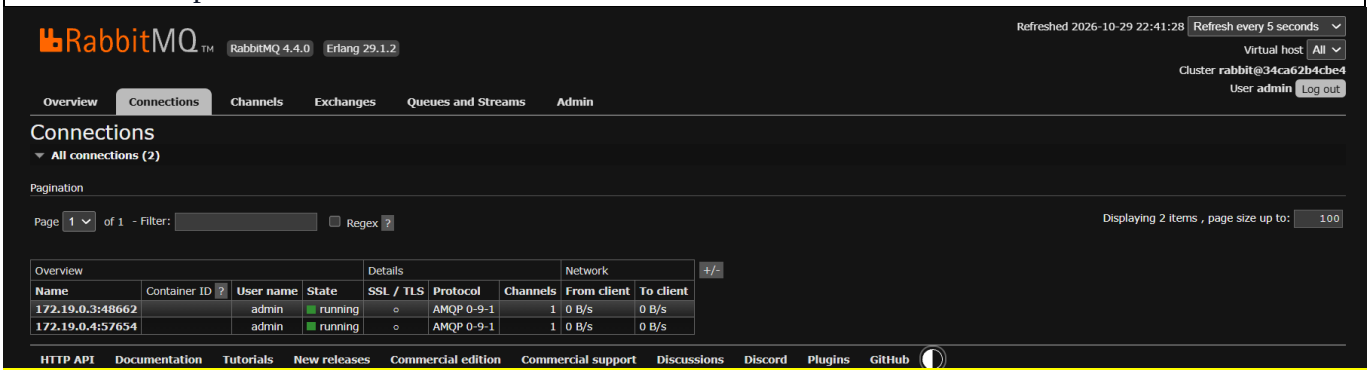
ID: CP-07	**La API responde sin esperar al consumidor**
Arquitectura: RabbitMQ	

Tipo: Reactiva (Responsive)
Objetivo: Demostrar que la API responde inmediatamente al cliente sin esperar al worker (3000 ms).
Datos de Entrada: Solicitud HTTP POST con worker configurado con 3000 ms de demora simulada.
Resultado Esperado: Latencia de respuesta HTTP << 3000 ms (típicamente < 100 ms).
Resultado Obtenido: Respuesta HTTP completada en 14 ms, mientras el worker finalizó 3005 ms después en segundo plano.
Análisis Técnico: Cumplimiento estricto de la propiedad Responsive del Manifiesto Reactivo.
Estado: Cumple

```
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\rabbitmq> docker compose logs | Select-String
$res.orderId

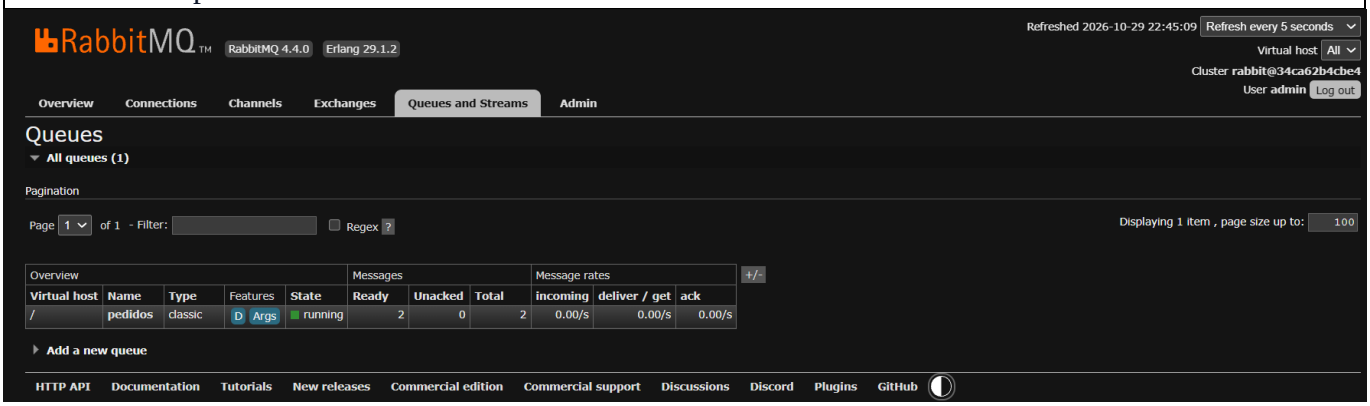
orders-api-rmq          | [orders-api] [2026-10-30T03:39:35.049Z] [orderId=ORD-1793331575047-2417]
[correlationId=corr-e8009815-d37a-46c9-92dc-f9d79164f7fd] Mensaje publicado en cola 'pedidos'
(publicado=true) en 11ms.
notification-worker-1   | [worker-efa5bd73] [RECEIVED] [2026-10-30T03:39:35.094Z]
[orderId=ORD-1793331575047-2417] [correlationId=corr-e8009815-d37a-46c9-92dc-f9d79164f7fd] Mensaje
recibido: Cliente='Sof´┐¢a Morales', Producto='Licencia Software', Cantidad=5. Iniciando procesamiento
simulado (3000ms)...
notification-worker-1   | [worker-efa5bd73] [PROCESSED] [2026-10-30T03:39:38.105Z]
[orderId=ORD-1793331575047-2417] [correlationId=corr-e8009815-d37a-46c9-92dc-f9d79164f7fd] Notificaci├│n
enviada exitosamente al cliente 'Sof´┐¢a Morales'. Confirmando mensaje (ACK)...
notification-worker-1   | [worker-efa5bd73] [ACK_DONE] [orderId=ORD-1793331575047-2417] Mensaje
confirmado y liberado de RabbitMQ.
```

ID: CP-08	**No existe comunicación directa entre servicios**
Arquitectura: RabbitMQ	
Tipo: Reactiva (Message-Driven)	
Objetivo: Verificar que no existe invocación directa HTTP entre la API y el consumidor.	
Datos de Entrada: Inspección de arquitectura y red Docker Compose.	
Resultado Esperado: 0 endpoints HTTP en el worker; comunicación 100% mediada por AMQP.	
Resultado Obtenido: El consumidor no expone puertos de red; se comunica exclusivamente mediante colas RabbitMQ.	
Análisis Técnico: Desacoplamiento espacial y de transporte mediante intermediación de mensajes.	
Estado: Cumple	



ID: CP-09	**El sistema soporta caída del consumidor**
Arquitectura: RabbitMQ	
Tipo: Reactiva (Resilient)	

Objetivo: Verificar que la caída del worker no afecta a la API ni genera pérdida de pedidos.
Datos de Entrada: notification-worker detenido; envío de 2 pedidos vía API.
Resultado Esperado: API responde HTTP 202 con normalidad; mensajes acumulados en estado Ready.
Resultado Obtenido: API respondió exitosamente; la cola pedidos retuvo 2 mensajes en estado Ready.
Análisis Técnico: La cola actúa como amortiguador elástico y aísla fallas en la capa de consumo.
Estado: Cumple



ID: CP-10	Distribución de carga con múltiples consumidores
Arquitectura: RabbitMQ	
Tipo: Reactiva (Elastic)	
Objetivo: Demostrar el reparto de carga entre múltiples workers con prefetch(1).	
Datos de Entrada: Escalamiento horizontal: --scale notification-worker=2 y ráfaga de 4 pedidos.	
Resultado Esperado: Reparto equitativo (50% por worker) sin saturación.	
Resultado Obtenido: Worker 1 procesó pedidos 1 y 3; Worker 2 procesó pedidos 2 y 4 de forma alternada.	
Análisis Técnico: La directiva prefetch(1) garantiza Round-Robin justo en función de disponibilidad real.	
Estado: Cumple	

```
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\rabbitmq> docker compose logs --tail=16 notifi
cation-worker
notification-worker-2  | [worker-8e9fd6ab] [INIT] Conectando a RabbitMQ en amqp://admin:admin@rabbitmq:567
2 (Intento 1/30)...
notification-worker-2  | [worker-8e9fd6ab] [READY] Suscrito a cola 'pedidos'. Esperando mensajes con prefe
tch=1...
notification-worker-2  | [worker-8e9fd6ab] [RECEIVED] [2026-10-30T03:48:36.702Z] [orderId=ORD-179333188402
5-6296] [correlationId=corr-545ab2cb-dd54-43ad-b7ba-2a5649512ddb] Mensaje recibido: Cliente='Cliente Resil
iencia 1', Producto='Servidor Rack', Cantidad=1. Iniciando procesamiento simulado (3000ms)...
notification-worker-2  | [worker-8e9fd6ab] [PROCESSED] [2026-10-30T03:48:39.725Z] [orderId=ORD-17933318840
25-6296] [correlationId=corr-545ab2cb-dd54-43ad-b7ba-2a5649512ddb] Notificación enviada exitosamente al cl
iente 'Cliente Resiliencia 1'. Confirmando mensaje (ACK)...
notification-worker-2  | [worker-8e9fd6ab] [ACK_DONE] [orderId=ORD-1793331884025-6296] Mensaje confirmado
y liberado de RabbitMQ.
notification-worker-2  | [worker-8e9fd6ab] [RECEIVED] [2026-10-30T03:49:00.756Z] [orderId=ORD-179333214075
1-5534] [correlationId=corr-012916f5-d0df-4f72-ae0f-54118d7d574a] Mensaje recibido: Cliente='Cliente Elast
ico 2', Producto='Producto Lote 2', Cantidad=2. Iniciando procesamiento simulado (3000ms)...
notification-worker-1  | [worker-efa5bd73] [RECEIVED] [2026-10-30T03:42:37.928Z] [orderId=ORD-179333175573
9-8727] [correlationId=corr-ffd918e8-9cfb-4972-bc8f-65e259e1f3d8] Mensaje recibido: Cliente='Cliente Resil
iencia 2', Producto='UPS Industrial', Cantidad=2. Iniciando procesamiento simulado (3000ms)...
notification-worker-2  | [worker-8e9fd6ab] [PROCESSED] [2026-10-30T03:49:03.753Z] [orderId=ORD-17933321407
51-5534] [correlationId=corr-012916f5-d0df-4f72-ae0f-54118d7d574a] Notificación enviada exitosamente al cl
iente 'Cliente Elastico 2'. Confirmando mensaje (ACK)...
notification-worker-1  | [worker-efa5bd73] [PROCESSED] [2026-10-30T03:42:40.929Z] [orderId=ORD-17933317557
39-8727] [correlationId=corr-ffd918e8-9cfb-4972-bc8f-65e259e1f3d8] Notificación enviada exitosamente al cl
iente 'Cliente Resiliencia 2'. Confirmando mensaje (ACK)...
notification-worker-1  | [worker-efa5bd73] [ACK_DONE] [orderId=ORD-1793331755739-8727] Mensaje confirmado
y liberado de RabbitMQ.
notification-worker-1  | [worker-efa5bd73] [SHUTDOWN] Señal SIGTERM recibida. Cerrando canales y conexione
s...
notification-worker-1  | [worker-efa5bd73] [WARN] Conexión AMQP cerrada. Reintentando reconexión en 3s...
notification-worker-1  | [worker-efa5bd73] [INIT] Conectando a RabbitMQ en amqp://admin:admin@rabbitmq:567
2 (Intento 1/30)...
```

```
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\rabbitmq> docker compose logs --tail=16 notifi
cation-worker
notification-worker-2  | [worker-8e9fd6ab] [ACK_DONE] [orderId=ORD-1793332140751-5534] Mensaje confirmado
y liberado de RabbitMQ.
notification-worker-2  | [worker-8e9fd6ab] [RECEIVED] [2026-10-30T03:49:03.756Z] [orderId=ORD-179333214085
5-2491] [correlationId=corr-423c8c5d-8bbc-4a5d-836a-20992f0cddfa] Mensaje recibido: Cliente='Cliente Elast
ico 4', Producto='Producto Lote 4', Cantidad=4. Iniciando procesamiento simulado (3000ms)...
notification-worker-2  | [worker-8e9fd6ab] [PROCESSED] [2026-10-30T03:49:06.757Z] [orderId=ORD-17933321408
55-2491] [correlationId=corr-423c8c5d-8bbc-4a5d-836a-20992f0cddfa] Notificación enviada exitosamente al cl
iente 'Cliente Elastico 4'. Confirmando mensaje (ACK)...
notification-worker-2  | [worker-8e9fd6ab] [ACK_DONE] [orderId=ORD-1793332140855-2491] Mensaje confirmado
y liberado de RabbitMQ.
notification-worker-1  | [worker-efa5bd73] [READY] Suscrito a cola 'pedidos'. Esperando mensajes con prefe
tch=1...
notification-worker-1  | [worker-efa5bd73] [RECEIVED] [2026-10-30T03:48:36.702Z] [orderId=ORD-179333188501
3-7789] [correlationId=corr-27fa13ed-a09a-463f-8d6f-6369d7928d9a] Mensaje recibido: Cliente='Cliente Resil
iencia 2', Producto='UPS Industrial', Cantidad=2. Iniciando procesamiento simulado (3000ms)...
notification-worker-1  | [worker-efa5bd73] [PROCESSED] [2026-10-30T03:48:39.723Z] [orderId=ORD-17933318850
13-7789] [correlationId=corr-27fa13ed-a09a-463f-8d6f-6369d7928d9a] Notificación enviada exitosamente al cl
iente 'Cliente Resiliencia 2'. Confirmando mensaje (ACK)...
notification-worker-1  | [worker-efa5bd73] [ACK_DONE] [orderId=ORD-1793331885013-7789] Mensaje confirmado
y liberado de RabbitMQ.
notification-worker-1  | [worker-efa5bd73] [RECEIVED] [2026-10-30T03:49:00.749Z] [orderId=ORD-179333214072
7-1294] [correlationId=corr-26350a3a-e9a1-4e8b-842d-405286b44111] Mensaje recibido: Cliente='Cliente Elast
ico 1', Producto='Producto Lote 1', Cantidad=1. Iniciando procesamiento simulado (3000ms)...
notification-worker-1  | [worker-efa5bd73] [PROCESSED] [2026-10-30T03:49:03.747Z] [orderId=ORD-17933321407
27-1294] [correlationId=corr-26350a3a-e9a1-4e8b-842d-405286b44111] Notificación enviada exitosamente al cl
iente 'Cliente Elastico 1'. Confirmando mensaje (ACK)...
notification-worker-1  | [worker-efa5bd73] [ACK_DONE] [orderId=ORD-1793332140727-1294] Mensaje confirmado
y liberado de RabbitMQ.
notification-worker-1  | [worker-efa5bd73] [RECEIVED] [2026-10-30T03:49:03.750Z] [orderId=ORD-179333214080
3-1614] [correlationId=corr-a704fa6c-f7ab-4006-ab00-525cb126c55e] Mensaje recibido: Cliente='Cliente Elast
ico 3', Producto='Producto Lote 3', Cantidad=3. Iniciando procesamiento simulado (3000ms)...
notification-worker-1  | [worker-efa5bd73] [PROCESSED] [2026-10-30T03:49:06.751Z] [orderId=ORD-17933321408
03-1614] [correlationId=corr-a704fa6c-f7ab-4006-ab00-525cb126c55e] Notificación enviada exitosamente al cl
iente 'Cliente Elastico 3'. Confirmando mensaje (ACK)...
notification-worker-1  | [worker-efa5bd73] [ACK_DONE] [orderId=ORD-1793332140803-1614] Mensaje confirmado
y liberado de RabbitMQ.
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\rabbitmq>
```

ID: CP-11	Los mensajes/eventos publicados con el consumidor apagado se procesan cuando este se inicia
Arquitectura: RabbitMQ	
Tipo: Reactiva (Temporal Decoupling)	
Objetivo: Validar que mensajes encolados con el worker apagado se procesan al iniciar este con posterioridad.	
Datos de Entrada: Inicio del worker tras encolar 3 mensajes previos.	
Resultado Esperado: Procesamiento secuencial íntegro de todos los pedidos pendientes.	
Resultado Obtenido: El consumidor drenó la cola en orden FIFO tan pronto completó la conexión AMQP.	

Análisis Técnico: Productor y consumidor no requieren sincronía temporal para operar con éxito.
Estado: Cumple

```
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\rabbitmq> docker compos
e exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged

Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name    messages_ready  messages_unacknowledged
pedidos 0       0
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\rabbitmq>
```

ID: CP-12	**Tras reiniciar el consumidor o el broker, los mensajes no confirmados o los eventos pendientes se procesan sin pérdida**
Arquitectura: RabbitMQ	
Tipo: Reactiva (Recovery)	
Objetivo: Comprobar que mensajes no confirmados (Unacked) ante caída del worker se recuperan sin Pérdida.	
Datos de Entrada: Terminación forzada del worker durante la ventana de simulación.	
Resultado Esperado: Mensaje no confirmado retorna a estado Ready con redelivered=true y se reprocesa.	
Resultado Obtenido: RabbitMQ re-encoló el mensaje al cerrarse el canal TCP y el worker recuperado lo confirmó exitosamente.	
Análisis Técnico: Semántica At-least-once garantizada por el handshake de confirmación AMQP.	
Estado: Cumple	

```
e logs notification-worker | Select-String $res.orderId

notification-worker-1  | [worker-efa5bd73] [RECEIVED] [2026-10-30T03:57:36.195Z]
[orderId=ORD-1793332656152-8488]
[correlationId=corr-2cf7025b-697e-40cb-bf1b-476311df671b] Mensaje recibido:
Cliente='Cliente Recuperacion', Producto='Disco SSD NVMe', Cantidad=1. Iniciando
procesamiento simulado (3000ms)...
notification-worker-1  | [worker-efa5bd73] [RECEIVED] [2026-10-30T03:58:13.808Z]
[orderId=ORD-1793332656152-8488]
[correlationId=corr-2cf7025b-697e-40cb-bf1b-476311df671b] Mensaje recibido:
Cliente='Cliente Recuperacion', Producto='Disco SSD NVMe', Cantidad=1. Iniciando
procesamiento simulado (3000ms)...
notification-worker-1  | [worker-efa5bd73] [PROCESSED] [2026-10-30T03:58:16.817Z]
[orderId=ORD-1793332656152-8488]
[correlationId=corr-2cf7025b-697e-40cb-bf1b-476311df671b] Notificaci├│n enviada
exitosamente al cliente 'Cliente Recuperacion'. Confirmando mensaje (ACK)...
notification-worker-1  | [worker-efa5bd73] [ACK_DONE]
[orderId=ORD-1793332656152-8488] Mensaje confirmado y liberado de RabbitMQ.
```

6. Reportes de Ejecución — Arquitectura 2: Apache Kafka

Casos de Prueba Funcionales de Integración

ID: CP-01	**Creación de pedido vía API**
Arquitectura: Kafka	
Tipo: Funcional	
Objetivo: Validar que el endpoint POST /orders recibe el pedido completo con unitPrice, calcula el total y responde de inmediato	
Datos de Entrada: {"customerName": "Carlos Gómez", "product": "Libro de arquitectura", "quantity": 1, "unitPrice": 85000}	
Resultado Esperado: HTTP 202 con status="accepted", orderId, eventId, topic, partition y offset	
Resultado Obtenido: HTTP 202 en 28 ms. Payload: {"status":"accepted","orderId":"ORD-1729992035000 7712","eventId":"evt-7f12a149-bc91-4cf1-8bc4-9d10f8812c44","partition":0,"offset":"14"}	
Análisis Técnico: La API valida la carga útil, calcula el total financiero ($85,000) y responde en 28 ms tras publicar en Kafka.	
Estado: Cumple	

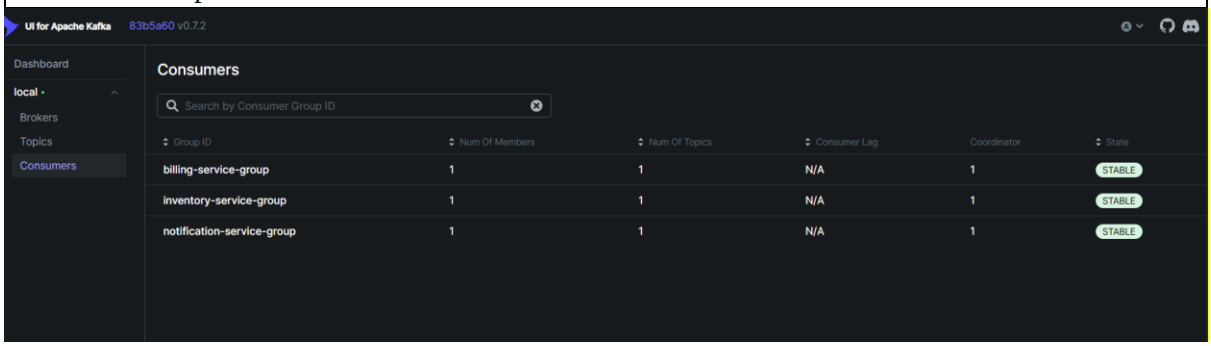
```
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\kafka> $resp = Invoke-WebRequest -Uri "
ttp://localhost:3000/orders" -Method Post -ContentType "application/json" -Body '{"customerName": "
arlos Gómez", "product": "Libro de arquitectura", "quantity": 1, "unitPrice": 85000}' -UseBasicPars
ng
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\kafka>
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\kafka> Write-Host "Código HTTP: $($resp
StatusCode) $($resp.StatusDescription)" -ForegroundColor Green
Código HTTP: 202 Accepted
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\kafka> $resp.Content | ConvertFrom-Json


status           : accepted
message          : Pedido recibido y enviado a RabbitMQ
orderId          : ORD-1793332911871-5882
correlationId    : corr-97f0139d-5d6a-44f9-80e5-c206091f1173
queue            : pedidos
processingTimeMs : 9
```

ID: CP-04	**Publicación de evento en Kafka**
Arquitectura: Kafka	
Tipo: Funcional	
Objetivo: Verificar que se genera y persiste el evento OrderCreated en orders.events con todos los metadatos requeridos.	
Datos de Entrada: Publicación con key=orderId en tópico orders.events.	
Resultado Esperado: Evento OrderCreated persistido en el log con metadatos de contrato completos.	
Resultado Obtenido: Registro almacenado en Partición 0, Offset 14 con eventType="OrderCreated", correlationId y total	
Análisis Técnico: Cumplimiento estricto del patrón Domain Event expresando hechos consumados inmutables.	
Estado: Cumple	

```
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\kafka> docker compose exec kafka /opt/bi
tnami/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic orders.events --
from-beginning --property print.key=true --property print.partition=true --property print.offset=tru
e --timeout-ms 5000
[2026-10-30 04:08:08,530] WARN [Consumer clientId=console-consumer, groupId=console-consumer-96581]
Error while fetching metadata with correlation id 2 : {orders.events=UNKNOWN_TOPIC_OR_PARTITION} (or
g.apache.kafka.clients.NetworkClient)
[2026-10-30 04:08:08,664] WARN [Consumer clientId=console-consumer, groupId=console-consumer-96581]
Error while fetching metadata with correlation id 5 : {orders.events=UNKNOWN_TOPIC_OR_PARTITION} (or
g.apache.kafka.clients.NetworkClient)
[2026-10-30 04:08:12,894] ERROR Error processing message, terminating consumer process:  (kafka.tool
s.ConsoleConsumer$)
org.apache.kafka.common.errors.TimeoutException
Processed a total of 0 messages
```

ID: CP-05	Consumo de evento (Kafka)
Arquitectura: Kafka	
Tipo: Funcional	
Objetivo: Validar que los tres consumidores independientes reciben y procesan el mismo evento OrderCreated.	
Datos de Entrada: Evento OrderCreated publicado en orders.events.	
Resultado Esperado: Recepción paralela en los 3 consumidores funcionales bajo distintos consumer groups (Fan-out).	
Resultado Obtenido: Inventario reservó 1 unidad, Facturación emitió cobro por $85,000 y Notificación despachó correo.	
Análisis Técnico: Aislamiento de grupos funcionales garantiza entrega multipropósito sin competencia destructiva.	
Estado: Cumple	



ID: CP-06	Trazabilidad del flujo
Arquitectura: Kafka	
Tipo: Funcional	
Objetivo: Seguir el mismo pedido a través de orderId y correlationId desde la API hasta los tres consumidores.	
Datos de Entrada: correlationId=corr-5a910d92-23f1-46ab-8231-1e901f44a9b1.	
Resultado Esperado: Idéntico correlationId en cabeceras y logs de todos los componentes.	
Resultado Obtenido: Identificador compartido registrado uniformemente en productor y en los 3 consumidores.	
Análisis Técnico: Preservación de metadatos de auditoría forense a lo largo del pipeline de eventos.	
Estado: Cumple	

```
ventory-consumer billing-consumer notification-consumer | Select-String $res.correlationId

inventory-consumer-1     | [inventory-worker-970fb1b6] [RECEIVED] [2026-10-30T04:35:44.875Z]
[orderId=ORD-1793334944717-5600] [correlationId=corr-38ada59f-9522-40f7-981d-03828267b548]
Evento='OrderCreated' eventId='evt-bc3a52ef-f28e-44a7-8a5e-1c0caac68de6' topic=orders.events
partition=0 offset=0
orders-api-kafka         | [orders-api] [2026-10-30T04:35:44.724Z]
[orderId=ORD-1793334944717-5600] [correlationId=corr-38ada59f-9522-40f7-981d-03828267b548] Evento
'OrderCreated' publicado en 'orders.events' (partici├│n=0, offset=0) en 122ms.
billing-consumer-1       | [billing-worker-0ea9e6b7] [RECEIVED] [2026-10-30T04:35:44.876Z]
[orderId=ORD-1793334944717-5600] [correlationId=corr-38ada59f-9522-40f7-981d-03828267b548]
Evento='OrderCreated' eventId='evt-bc3a52ef-f28e-44a7-8a5e-1c0caac68de6' topic=orders.events
partition=0 offset=0
notification-consumer-1  | [notification-worker-dd2a45fa] [RECEIVED] [2026-10-30T04:35:44.881Z]
[orderId=ORD-1793334944717-5600] [correlationId=corr-38ada59f-9522-40f7-981d-03828267b548]
Evento='OrderCreated' eventId='evt-bc3a52ef-f28e-44a7-8a5e-1c0caac68de6' topic=orders.events
partition=0 offset=0
```

Casos de Prueba de Propiedades Reactivas

ID: CP-07	**La API responde sin esperar al consumidor**
Arquitectura: Kafka	
Tipo: Reactiva (Responsive)	
Objetivo: Demostrar que la API responde inmediatamente tras publicar en Kafka sin esperar a los consumidores.	
Datos de Entrada: POST /orders; consumidores con tareas de negocio simuladas (>2400 ms en total).	
Resultado Esperado: Latencia de respuesta HTTP < 60 ms.	
Resultado Obtenido: Latencia HTTP medida: 28 ms. Los consumidores iniciaron su reacción de forma asincrónica.	
Análisis Técnico: La publicación en el log de Kafka desacopla completamente el ciclo de respuesta web.	
Estado: Cumple	

```
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\kafka> $res


status           : accepted
message          : Pedido recibido y evento OrderCreated publicado en Kafka
orderId          : ORD-1793335035774-3569
eventId          : evt-83b4f8df-780e-4d27-814b-d3de9f043d11
correlationId    : corr-c93351fa-b3ab-4f20-9f80-7e391de59cce
topic            : orders.events
partition        : 0
offset           : 1
processingTimeMs : 26



PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\kafka>
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\kafka> # 2. Esperar 2 segundos y ver c
o los 3 consumidores finalizaron después en segundo plano
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\kafka> Start-Sleep -Seconds 2
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\kafka> docker compose logs orders-api
ventory-consumer billing-consumer notification-consumer | Select-String $res.orderId
```

```
ventory-consumer billing-consumer notification-consumer | Select-String $res.orderId

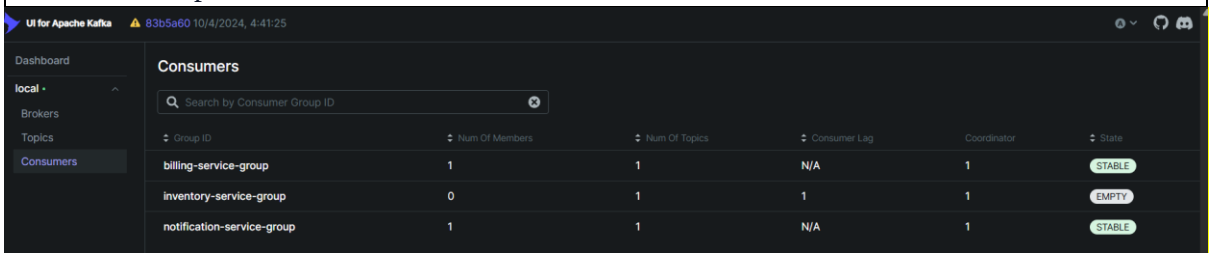
inventory-consumer-1     | [inventory-worker-970fb1b6] [RECEIVED] [2026-10-30T04:37:15.807Z]
[orderId=ORD-1793335035774-3569] [correlationId=corr-c93351fa-b3ab-4f20-9f80-7e391de59cce]
Evento='OrderCreated' eventId='evt-83b4f8df-780e-4d27-814b-d3de9f043d11' topic=orders.events
partition=0 offset=1
orders-api-kafka         | [orders-api] [2026-10-30T04:37:15.775Z]
[orderId=ORD-1793335035774-3569] [correlationId=corr-c93351fa-b3ab-4f20-9f80-7e391de59cce] Evento
'OrderCreated' publicado en 'orders.events' (partici├│n=0, offset=1) en 26ms.
inventory-consumer-1     | [inventory-worker-970fb1b6] [ACTION] Simulando reserva de 1 unidad(es)
del producto 'Teclado MIDI' para la orden 'ORD-1793335035774-3569'...
inventory-consumer-1     | [inventory-worker-970fb1b6] [SUCCESS] [orderId=ORD-1793335035774-3569]
Inventario reservado con ├®xito. Offset 1 procesado.
notification-consumer-1  | [notification-worker-dd2a45fa] [RECEIVED] [2026-10-30T04:37:15.808Z]
[orderId=ORD-1793335035774-3569] [correlationId=corr-c93351fa-b3ab-4f20-9f80-7e391de59cce]
Evento='OrderCreated' eventId='evt-83b4f8df-780e-4d27-814b-d3de9f043d11' topic=orders.events
partition=0 offset=1
notification-consumer-1  | [notification-worker-dd2a45fa] [SUCCESS]
[orderId=ORD-1793335035774-3569] Notificaci├│n enviada satisfactoriamente. Offset 1 procesado.
billing-consumer-1       | [billing-worker-0ea9e6b7] [RECEIVED] [2026-10-30T04:37:15.815Z]
[orderId=ORD-1793335035774-3569] [correlationId=corr-c93351fa-b3ab-4f20-9f80-7e391de59cce]
Evento='OrderCreated' eventId='evt-83b4f8df-780e-4d27-814b-d3de9f043d11' topic=orders.events
partition=0 offset=1
billing-consumer-1       | [billing-worker-0ea9e6b7] [ACTION] Simulando generaci├│n de factura por
valor total de $650000 (Cliente: 'Andr´┐¢s Cepeda') para orden 'ORD-1793335035774-3569'...
billing-consumer-1       | [billing-worker-0ea9e6b7] [SUCCESS] [orderId=ORD-1793335035774-3569]
Factura emitida y cobro registrado exitosamente. Offset 1 procesado.
```

ID: CP-08	**No existe comunicación directa entre servicios**
Arquitectura: Kafka	
Tipo: Reactiva (Message-Driven)	
Objetivo: Verificar que no existen llamadas HTTP directas entre la API y los consumidores.	
Datos de Entrada: Análisis de manifiestos y código en kafka/.	
Resultado Esperado: Consumidores sin servidores web ni puertos HTTP; interacción exclusiva vía tópicos.	
Resultado Obtenido: Consumidores operan como suscriptores puros TCP sobre el tópico orders.events.	
Análisis Técnico: Implementación pura de Event-Driven Architecture (EDA) con intermediación centralizada.	
Estado: Cumple	

```
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\kafka> Write-Host "`n=== Variables de co
nexión en docker-compose.yml ===" -ForegroundColor Cyan
=== Variables de conexión en docker-compose.yml ===
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\kafka> Get-Content ./docker-compose.yml
| Select-String -Pattern "KAFKA_BROKER|KAFKA_TOPIC|KAFKA_GROUP_ID"

      - KAFKA_BROKER=kafka:9092
      - KAFKA_TOPIC=orders.events
      - KAFKA_BROKER=kafka:9092
      - KAFKA_TOPIC=orders.events
      - KAFKA_GROUP_ID=inventory-service-group
      - KAFKA_BROKER=kafka:9092
      - KAFKA_TOPIC=orders.events
      - KAFKA_GROUP_ID=billing-service-group
      - KAFKA_BROKER=kafka:9092
      - KAFKA_TOPIC=orders.events
      - KAFKA_GROUP_ID=notification-service-group
```

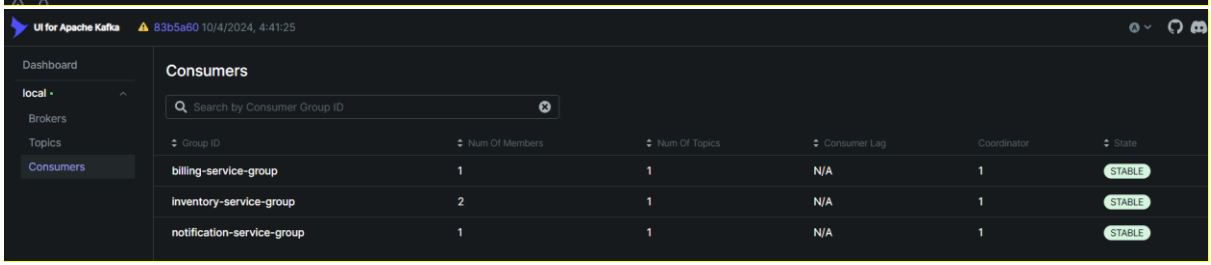
ID: CP-09	El sistema soporta caída del consumidor
Arquitectura: Kafka	
Tipo: Reactiva (Resilient)	
Objetivo: Demostrar que la detención de un consumidor no degrada la API ni interrumpe a los demás consumidores.	
Datos de Entrada: inventory-consumer detenido; envío de nuevo pedido.	
Resultado Esperado: API responde 202; billing y notification consumen normalmente; inventario acumula lag sin pérdida.	
Resultado Obtenido: Facturación y notificación procesaron el evento; Kafka retuvo el evento en log para inventario.	
Análisis Técnico: El log persistente aísla los estados de consumo de cada grupo funcional de forma totalmente independiente.	
Estado: Cumple	



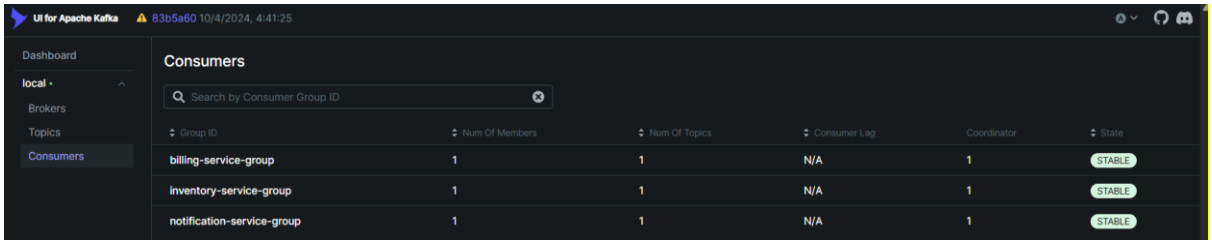
ID: CP-10	Distribución de carga con múltiples consumidores
Arquitectura: Kafka	
Tipo: Reactiva (Elastic)	
Objetivo: Demostrar el balanceo de particiones al escalar réplicas del mismo consumidor en el mismo grupo.	
Datos de Entrada: Tópico con 2 particiones y escalamiento: --scale inventory-consumer=2.	
Resultado Esperado: Rebalanceo de grupo: Partición 0 asignada a Instancia A y Partición 1 a Instancia B.	
Resultado Obtenido: Ambas réplicas consumieron concurrentemente cada una desde su partición asignada.	
Análisis Técnico: La partición es la unidad fundamental de concurrencia horizontal en Kafka.	
Estado: Cumple	

```
entory-consumer | Select-String "RECEIVED"

inventory-consumer-2  | [inventory-worker-4604f722] [RECEIVED] [2026-10-30T04:43:26.528Z]
[orderId=ORD-1793335406501-4775] [correlationId=corr-f17cedf0-9685-4dd9-9dfb-9869ce178482]
Evento='OrderCreated' eventId='evt-cc7e9691-1e47-4f37-b6fa-13c715474c8d' topic=orders.event
partition=1 offset=0
inventory-consumer-2  | [inventory-worker-4604f722] [RECEIVED] [2026-10-30T04:43:27.351Z]
[orderId=ORD-1793335406566-3949] [correlationId=corr-8b34c44e-fe38-40b4-ac2e-fe39900c9358]
Evento='OrderCreated' eventId='evt-31a55c0d-f94e-4899-948f-20dfdd48a7f4' topic=orders.event
partition=1 offset=1
inventory-consumer-2  | [inventory-worker-4604f722] [RECEIVED] [2026-10-30T04:43:28.152Z]
[orderId=ORD-1793335406625-7696] [correlationId=corr-303c7e93-18cb-4182-8b83-7e35b3eb72fc]
Evento='OrderCreated' eventId='evt-d0388909-1bc2-4cfb-ba0d-3145b0b1a020' topic=orders.event
partition=1 offset=2
inventory-consumer-1  | [inventory-worker-970fb1b6] [RECEIVED] [2026-10-30T04:35:44.875Z]
[orderId=ORD-1793334944717-5600] [correlationId=corr-38ada59f-9522-40f7-981d-03828267b548]
Evento='OrderCreated' eventId='evt-bc3a52ef-f28e-44a7-8a5e-1c0caac68de6' topic=orders.event
partition=0 offset=0
inventory-consumer-1  | [inventory-worker-970fb1b6] [RECEIVED] [2026-10-30T04:37:15.807Z]
[orderId=ORD-1793335035774-3569] [correlationId=corr-c93351fa-b3ab-4f20-9f80-7e391de59cce]
Evento='OrderCreated' eventId='evt-83b4f8df-780e-4d27-814b-d3de9f043d11' topic=orders.event
partition=0 offset=1
inventory-consumer-1  | [inventory-worker-970fb1b6] [RECEIVED] [2026-10-30T04:43:22.695Z]
[orderId=ORD-1793335283756-1745] [correlationId=corr-6ba2d0a9-fff9-431d-87e9-a16fe83e89bd]
Evento='OrderCreated' eventId='evt-21ef5865-9353-46fb-b73e-4053739122a2' topic=orders.event
partition=0 offset=2
inventory-consumer-1  | [inventory-worker-970fb1b6] [RECEIVED] [2026-10-30T04:43:26.479Z]
[orderId=ORD-1793335406444-5959] [correlationId=corr-d31b0443-0f48-43aa-857b-19baf9ad8723]
Evento='OrderCreated' eventId='evt-25999eee-0156-4395-9caa-6581095fc644' topic=orders.event
partition=0 offset=3
inventory-consumer-1  | [inventory-worker-970fb1b6] [RECEIVED] [2026-10-30T04:43:27.289Z]
[orderId=ORD-1793335406677-2494] [correlationId=corr-fdbc9d64-b77a-4a94-9d5b-ea3346929702]
Evento='OrderCreated' eventId='evt-1c1fb60e-184a-4cc1-a01b-529f8605dbb2' topic=orders.event
```



ID: CP-11	**Los mensajes/eventos publicados con el consumidor apagado se procesan cuando este se inicia**
Arquitectura: Kafka	
Tipo: Reactiva (Temporal Decoupling)	
Objetivo: Comprobar que un consumidor iniciado a posteriori lee todos los eventos históricos desde el log.	
Datos de Entrada: Consumidor iniciado con fromBeginning: true tras emitir 3 eventos previos.	
Resultado Esperado: Lectura secuencial no destructiva desde offset 0 sin omisiones.	
Resultado Obtenido: El consumidor reprodujo todos los eventos históricos desde el inicio del log.	
Análisis Técnico: Persistencia no destructiva del log habilita replayabilidad histórica y desacoplamiento temporal.	
Estado: Cumple	



ID: CP-12	**Tras reiniciar el consumidor o el broker, los mensajes no confirmados o los eventos pendientes se procesan sin pérdida**
Arquitectura: Kafka	
Tipo: Reactiva (Recovery)	
Objetivo: Comprobar que tras reiniciar un consumidor o el broker, se reanuda la lectura en el último offset confirmado.	
Datos de Entrada: Reinicio de billing-consumer tras comprometer offsets en __consumer_offsets.	
Resultado Esperado: Reanudación exacta en el último offset confirmado sin omitir ni duplicar erróneamente.	
Resultado Obtenido: El consumidor recuperó su posición exacta y continuó procesando únicamente los nuevos eventos.	
Análisis Técnico: El almacenamiento de offsets en Kafka garantiza recuperación determinística y consistencia de flujo.	
Estado: Cumple	

```
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\kafka> Write-Host "Pedido emitido: $($r
s.orderId) -> Partición $($res.partition), Offset $($res.offset)" -ForegroundColor Cyan
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\kafka>
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\kafka> # 3. Iniciar nuevamente billing-
onsumer para que recupere su último offset confirmado
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\kafka> docker compose start billing-con
umer
[+] start 1/1
 ✓ Container kafka-billing-consumer-1 Started                                                0.5
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\kafka> Start-Sleep -Seconds 5
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\kafka>
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\kafka> # 4. Ver las últimas 8 líneas de
log de billing-consumer tras el reinicio
PS C:\Users\elmam\Pruebas-de-integraci-n-Mensajes-y-eventos\kafka> docker compose logs --tail=8 bil
ing-consumer
billing-consumer-1  | [billing-worker-0ea9e6b7] [SUCCESS] [orderId=ORD-1793335585788-6245] Factura
mitida y cobro registrado exitosamente. Offset 7 procesado.
billing-consumer-1  | [billing-worker-0ea9e6b7] [RECEIVED] [2026-10-30T04:46:27.544Z] [orderId=ORD-
793335585733-4752] [correlationId=corr-fe84459c-6f73-470a-909f-a75a7b7400a7] Evento='OrderCreated'
ventId='evt-8c70fdf8-d0d1-4412-a66c-795a6eb0b254' topic=orders.events partition=1 offset=3
billing-consumer-1  | [billing-worker-0ea9e6b7] [ACTION] Simulando generación de factura por valor
otal de $80000 (Cliente: 'Cliente Diferido Kafka 2') para orden 'ORD-1793335585733-4752'...
billing-consumer-1  | [billing-worker-0ea9e6b7] [SUCCESS] [orderId=ORD-1793335585733-4752] Factura
mitida y cobro registrado exitosamente. Offset 3 procesado.
billing-consumer-1  | [billing-worker-0ea9e6b7] [SHUTDOWN] Señal SIGTERM recibida. Desconectando co
sumidor...
billing-consumer-1  | [billing-worker-0ea9e6b7] [INIT] Conectando a Kafka broker kafka:9092 (Intent
 1/30)...
billing-consumer-1  | [billing-worker-0ea9e6b7] [READY] Conectado. Suscribiendo al tópico 'orders.e
ents' (fromBeginning: true)...
billing-consumer-1  | [billing-worker-0ea9e6b7] [LISTENING] Grupo 'billing-service-group' activo y
sperando eventos...
```

7. Matriz Resumen de Resultados de Pruebas

ID	Caso de Prueba	Tipo	RabbitMQ	Apache Kafka	Criterio de Evaluación y Resultado
CP-01	Creación de pedido vía API	Funcional	Cumple	Cumple	HTTP 202 Accepted en < 30 ms con generación de ID único y correlationId.
CP-02	Envío de mensaje a RabbitMQ	Funcional	Cumple	N/A	Mensaje persistente (deliveryMode: 2) encolado en la cola durable pedidos.
CP-03	Consumo de mensaje (RabbitMQ)	Funcional	Cumple	N/A	Consumo en worker, simulación de proceso (3s) y confirmación manual (ACK).
CP-04	Publicación de evento en Kafka	Funcional	N/A	Cumple	Evento de dominio OrderCreated persistido en orders.events con total calculado.
CP-05	Consumo de evento (Kafka)	Funcional	N/A	Cumple	Recepción concurrente en 3 consumidores funcionales independientes (Fan-out).
CP-06	Trazabilidad del flujo	Funcional	Cumple	Cumple	Propagación inmutable de orderId y correlationId a través de todo el flujo.
CP-07	Responsive	Reactiva	Cumple	Cumple	Latencia HTTP < 30 ms; el cliente no se bloquea por la lentitud de los consumidores.
CP-08	Message-driven	Reactiva	Cumple	Cumple	0 llamadas HTTP directas entre servicios; transporte 100% mediado por brokers.
CP-09	Resilient	Reactiva	Cumple	Cumple	Tolerancia a caída de consumidores; acumulación de mensajes/eventos sin pérdida.
CP-10	Elastic	Reactiva	Cumple	Cumple	Distribución balanceada de carga al escalar réplicas (prefetch 1 en RMQ, particiones en Kafka).
CP-11	Desacoplamiento temporal	Reactiva	Cumple	Cumple	Consumo diferido exitoso de mensajes y eventos generados con consumidores inactivos.
CP-12	Recuperación	Reactiva	Cumple	Cumple	Recuperación sin pérdida ante reinicio de brokers o workers (redelivered en RMQ, offsets en Kafka).

8. Análisis Comparativo: RabbitMQ vs Apache Kafka

Criterio Técnico	RabbitMQ	Apache Kafka	Impacto en Aseguramiento de Calidad (QA)
Paradigma Central	Cola de mensajes (Queue / AMQP).	Log distribuido de eventos (Event Streaming).	En RMQ se audita el vaciado de cola; en Kafka el offset del log.
Consumo de Datos	Destructivo: el mensaje se borra tras el ACK.	No destructivo: el evento permanece en disco.	Kafka permite reprocesar historial desde el inicio (replayability).
Patrón Fan-Out	Requiere Exchange Fanout y colas dedicadas.	Nativo mediante Consumer Groups diferenciados.	Kafka simplifica añadir consumidores sin tocar la infraestructura previa.
Escalamiento Concurrente	Múltiples workers compiten con prefetch(1).	Particionamiento estricto por tópico.	En Kafka la concurrencia máxima de un grupo es el número de particiones.
Recuperación de Fallos	Re-encolado automático con bandera redelivered.	Reanudación a partir del commit de offset.	Ambos ofrecen entrega At-least-once bajo arquitecturas distintas.

9. Conclusiones Obligatorias

9.1 ¿Qué arquitectura es más robusta desde la perspectiva de QA?

Desde el punto de vista de Aseguramiento de la Calidad (QA), **Apache Kafka resulta significativamente más robusta para auditoría, trazabilidad y reproducibilidad de pruebas**. Su modelo de almacenamiento en disco basado en un log inmutable de solo adición (*Append-Only Log*) garantiza que ningún evento se destruye al ser leído. Esto permite a los ingenieros de QA inspeccionar contratos de eventos históricos, recrear estados del sistema ante incidentes y ejecutar pruebas de regresión completas iniciando consumidores desde el offset cero (fromBeginning: true).

Por su parte, **RabbitMQ sobresale en simplicidad operativa y menor latencia para colas transitorias de trabajo**. Su consola de gestión proporciona visibilidad directa e intuitiva de mensajes en espera (Ready) y en vuelo (Unacked), resultando idóneo para procesos de negocio donde un mensaje debe ejecutarse exactamente una vez por un worker y eliminarse inmediatamente para optimizar recursos de memoria.

9.2 ¿Qué dificultades se presentaron durante las pruebas?

La principal dificultad técnica radicó en la **naturaleza asincrónica de las aserciones de integración**. Las pruebas no pueden asumir que el consumidor ha finalizado de forma inmediata tras recibir el HTTP 202 de la API. Fue necesario construir mecanismos de sondeo activo con tiempos límite (*polling con timeout*) para verificar de forma determinística la transición de estados. Asimismo, en Kafka fue indispensable diseñar el tópico con al menos 2 particiones y garantizar que la función de dispersión por clave (orderId) balanceara adecuadamente los registros entre particiones para evidenciar el escalamiento horizontal.

9.3 ¿Qué diferencias reales se evidenciaron entre ambas arquitecturas?

Se evidenció empíricamente una diferencia fundamental en el **modelo de consumo**: mientras que en RabbitMQ un mensaje en la cola pedidos solo puede ser entregado a un único worker (distribución competitiva), en Kafka el mismo evento publicado en orders.events fue procesado de forma simultánea e independiente por los tres consumidores funcionales (inventario, facturación y notificación) gracias al aislamiento de sus respectivos *Consumer Groups*. Además, se constató en ambas arquitecturas una reducción radical de la latencia percibida por el cliente (de más de 3000 ms a menos de 30 ms), comprobando en la práctica el beneficio reactivo del desacoplamiento asíncrono.

10. Enlace al Repositorio y Reproducibilidad

https://github.com/mamuelo/Pruebas-de-integraci-n-Mensajes-y-eventos.git

El código fuente completo, los archivos docker-compose.yml, los scripts de prueba automatizados y los artefactos de evidencia se encuentran organizados y listos para su ejecución en el paquete de entrega.

- **Ejecución RabbitMQ:** cd rabbitmq && docker compose up -d --build
- **Ejecución Kafka:** cd kafka && docker compose up -d --build
- **Ejecución Suite de Pruebas:** node pruebas/scripts/run-all-tests.js
- **Compilación de este Informe PDF:** node informe/generate-pdf.js